TransitOps API Documentation

API Documentation

TransitOps REST API Reference

Version: 1.0 **Base URL:** `http://<host>/api/v1` **Interactive docs (Swagger UI):** `/api-docs` **Format:** JSON over HTTPS (HTTP in local dev) **Companion documents:** PRD, TRD, Backend Schema, SRS

---

1. Conventions

1.1 Authentication

All endpoints except `POST /auth/register`, `POST /auth/login`, `POST /auth/refresh`, `POST /auth/forgot-password`, and `POST /auth/reset-password` require:

```
Authorization: Bearer <JWT>
```

1.2 Standard Response Envelope

```
// Success
{ "success": true, "data": { ... } }

// Success (list)
{ "success": true, "count": 12, "data": [ ... ] }

// Error
{ "success": false, "message": "Human-readable error" }

// Validation error (Zod)
{
  "success": false,
  "message": "Validation failed",
  "errors": [ { "path": "cargoWeight", "message": "Cargo weight cannot be negative" } ]
}
```

In non-production environments, error responses also include a `stack` field.

1.3 Common Status Codes

Code	Meaning
200	Success
201	Resource created
400	Bad request / validation failure / business-rule violation
401	Missing, invalid, or expired JWT
403	Authenticated but role not permitted
404	Resource not found
429	Rate limit exceeded (100 req / 10 min / IP on `/api/*`)
500	Unhandled server error

1.4 Rate Limiting

100 requests per 10-minute window per IP address, applied to all `/api/*` routes.

---

2. Auth — `/api/v1/auth`

POST `/auth/register`

Access: Public · **Body validated by:** Zod `registerSchema`

Request body:

```
{
  "name": "Asha Rao",
  "email": "asha@example.com",
  "password": "SecurePass1",
  "phone": "+91 9876543210",
  "role": "dispatcher"
}
```

Field	Rule
`name`	string, ≥ 2 chars, required
`email`	valid email, required
`password`	string, ≥ 8 chars, required (hashed before storage)
`phone`	required
`role`	optional, one of `fleet_manager`, `dispatcher`, `safety_officer`, `financial_analyst` (`viewer` cannot self-register)

Response `201`:

```
{
  "success": true,
  "token": "<jwt>",
  "user": { "id": "665...", "name": "Asha Rao", "email": "asha@example.com", "role": "dispatcher" }
}
```

Error `400`: `{ "success": false, "message": "Email already registered" }`

POST `/auth/login`

Access: Public

Request body: `{ "email": "asha@example.com", "password": "SecurePass1" }`

Response `200`:

```
{ "success": true, "token": "<jwt>", "user": { "id": "665...", "name": "...", "email": "...", "role": "..." }
```

Error `401`: `{ "success": false, "message": "Invalid email or password" }`

POST `/auth/refresh`

Access: Public (requires a previously-issued, possibly-expired token in body)

Request body: `{ "token": "<expired-or-valid-jwt>" }`

Response `200`: `{ "success": true, "token": "<new-jwt>" }` Errors: `401` no token / invalid token; `404` user not found or deleted; `403` user suspended.

Note: token verification uses `ignoreExpiration: true`, so an expired-but-otherwise-valid token can still be refreshed.

POST `/auth/forgot-password`

Access: Public

Request body: `{ "email": "asha@example.com" }`

Response `200` (always, to avoid email enumeration):

```
{ "success": true, "message": "If that email is registered, a password reset link has been sent." }
```

Internally: signs a short-lived (15 min) JWT using `JWT_SECRET + currentPasswordHash` as the secret (so it self-invalidates on password change), and emails a reset link via nodemailer (falls back to an Ethereal test inbox if SMTP env vars are unset).

POST `/auth/reset-password`

Access: Public

Request body: `{ "id": "<userId>", "token": "<reset-token>", "password": "NewSecurePass1" }`

Response `200`: `{ "success": true, "message": "Password has been updated successfully. You may now log in." }`
Error `400`: invalid token/user, or expired/invalid token.

GET `/auth/me`

Access: Private (any authenticated role)

Response `200`: `{ "success": true, "user": { ...current user profile, no passwordHash... } }`

---

3. Vehicles — `/api/v1/vehicles`

Method	Path	Access	Cache
GET	`/`	Any authenticated	300s
POST	`/`	`fleet_manager`	invalidates `/vehicles*`
GET	`/:id`	Any authenticated	300s
PUT	`/:id`	`fleet_manager`	invalidates `/vehicles*`
DELETE	`/:id`	`fleet_manager`	invalidates `/vehicles*`

Validated by: Zod `VehicleSchema` (PUT uses `.partial()` — all fields optional).

Request body (create):

```
{
  "registrationNumber": "MH-12-AB-1234",
  "name": "Tata 1613 Truck",
  "vehicleType": "truck",
  "fuelType": "diesel",
  "maxLoadCapacity": 12000,
  "acquisitionCost": 2500000,
  "odometer": 0
}
```

Field	Rule
`registrationNumber`, `name`	required
`maxLoadCapacity`	required, ≥ 1
`vehicleType`	optional enum: `truck`, `van`, `bus`, `pickup`, `trailer`, `tanker`
`fuelType`	optional enum: `diesel`, `gasoline`, `cng`, `electric`, `hybrid`
`acquisitionCost`, `odometer`	optional, ≥ 0
`status`	optional enum: `available`, `on_trip`, `in_shop`, `retired`

Response `201` : `{ "success": true, "data": { "_id": "...", "registrationNumber": "...", "status": "available", ... } }` List response `200` : `{ "success": true, "data": [...], "total": 42, "page": 1, "pages": 3 }` (paginated via `BaseService.getAll`) Delete response `200` : `{ "success": true, "message": "Vehicle removed" }` (soft delete) Error `404` : `{ "success": false, "message": "Resource not found" }`

4. Drivers — `/api/v1/drivers`

Method	Path	Access
GET	`/` , `/:id`	Any authenticated
POST	`/` , PUT `/:id` , DELETE `/:id`	`safety_officer`

Validated by: Zod `DriverSchema` .

Request body (create):

```
{
  "name": "Rahul Kumar",
  "licenseNumber": "MH1420220012345",
  "licenseCategory": "C",
  "licenseExpiryDate": "2027-05-01",
  "contactNumber": "+91 9988776655",
  "paymentType": "monthly",
  "paymentAmount": 25000,
  "kyc": { "aadharNumber": "XXXX-XXXX-1234" }
}
```

Field	Rule
`name` , `licenseNumber`	required
`paymentAmount`	optional, ≥ 0
`safetyScore`	optional, 0–100
`status`	optional enum: `available` , `on_trip` , `on_leave` , `suspended` — ⚠ note: the persisted Mongoose schema instead defines this enum as `available` , `on_trip` , `off_duty` , `suspended` ; see Backend Schema doc §7 for this known discrepancy

`licenseNumber` and `contactNumber` are encrypted before persistence and transparently decrypted on read — the API always returns plaintext values to an authorized caller.

Response `201` : `{ "success": true, "data": { "_id": "...", "name": "...", "status": "available", ... } }` Query param supported on list: `?status=available` Error `404` : `{ "success": false, "message": "Driver not found" }`

5. Trips — `/api/v1/trips`

Method	Path	Access
GET	`/`	Any authenticated
POST	`/`	`dispatcher` , `fleet_manager`
GET	`/:id`	Any authenticated
PUT	`/:id`	`dispatcher` , `fleet_manager` (only while `draft`)
PUT	`/:id/dispatch`	`dispatcher` , `fleet_manager`
PUT	`/:id/complete`	`dispatcher` , `fleet_manager`
PUT	`/:id/cancel`	`dispatcher` , `fleet_manager`
PUT	`/:id/status`	`dispatcher` , `fleet_manager`

Validated by: Zod `TripSchema` (PUT uses `.partial()`).

GET `/trips` — query params

`status`, `priority`, `vehicle` (id), `driver` (id), `dateFrom`, `dateTo`, `search` (matches `tripCode`), `page`, `limit`.

Response `200`:

```
{ "success": true, "total": 87, "page": 1, "pages": 9, "data": [ { "_id": "...", "tripCode": "TRIP-A1B2C3D4", "status": "dispatched", "vehicle": "...", "driver": "...", "..." } ] }
```

POST `/trips`

Request body:

```
{
  "source": "Pune Warehouse",
  "destination": "Mumbai Port",
  "vehicle": "665f1a2b3c4d5e6f70819203",
  "driver": "665f1a2b3c4d5e6f70819204",
  "cargoWeight": 8500,
  "scheduledDate": "2026-08-01T06:00:00Z",
  "eta": "2026-08-01T12:00:00Z",
  "priority": "high"
}
```

Field	Rule
`source`, `destination`	required
`vehicle`, `driver`	required, must be valid 24-char hex ObjectId
`cargoWeight`	required, ≥ 0
`priority`	optional enum: `low`, `medium`, `high`, `urgent`

Response `201`: `{ "success": true, "data": { "tripCode": "TRIP-A1B2C3D4", "status": "draft", ... } }`

Error `400` examples (business-rule violations, thrown from `tripService.createTrip`):

```
{ "success": false, "message": "BR-1: Vehicle not found or is retired" }
{ "success": false, "message": "BR-4: Driver is suspended or license has expired" }
{ "success": false, "message": "BR-5: Cargo weight (8500kg) exceeds vehicle capacity (6000kg)" }
{ "success": false, "message": "Scheduling conflict: Vehicle already booked on trip TRIP-9F8E7D6C during this window" }
```

PUT `/trips/:id/dispatch`

No body required. Response `200`:

```
{ "success": true, "message": "Trip dispatched successfully", "data": { "status": "dispatched", ... } }
```

Error `400` if vehicle/driver not eligible (BR-3/BR-4/BR-5 re-checked at dispatch time).

PUT `/trips/:id/complete`

Request body:

```
{ "actualDistance": 148.5, "fuelConsumed": 42.0, "revenue": 18500 }
```

Response `200`: `{ "success": true, "message": "Trip completed successfully", "data": { "status": "completed", "completedAt": "...", ... } }` Side effects: vehicle → `available` (odometer += `actualDistance`), driver → `available`; may create a “Maintenance Due Soon” notification.

PUT `/trips/:id/cancel`

No body required. Response `200`: `{ "success": true, "message": "Trip cancelled", "data": { "status": "cancelled", ... } }` Error `400` if trip is not currently `draft` or `dispatched`.

PUT `/trips/:id/status`

Request body: `{ "status": "in_transit" }` Response `200`: `{ "success": true, "message": "Trip status updated", "data": { ... } }` Error `400` if the trip is already `completed` / `cancelled` (terminal states are locked).

---

6. Fuel & Expenses — `/api/v1/fuel-expenses`

Method	Path	Access
GET	`/live-rates`	Any authenticated
GET	`/fuel`	Any authenticated
POST	`/fuel`	`financial_analyst`, `fleet_manager`
GET	`/expenses`	Any authenticated
POST	`/expenses`	`financial_analyst`, `fleet_manager`

GET `/fuel-expenses/live-rates`

Response `200`:

```
{ "success": true, "data": { "diesel": 89.44, "gasoline": 96.51, "cng": 76.72, "electric": 15.02, "hybrid": 92.18 } }
```

Rates are a deterministic pseudo-random ±2% daily fluctuation over hardcoded base prices — not a real live feed.

GET `/fuel-expenses/fuel?vehicle=<id>`

Response `200`: `{ "success": true, "count": 5, "data": [ { "vehicle": {"name":"...", "registrationNumber":"..."}, "trip": {"tripCode":"..."}, "liters": 40, "cost": 3580, "station": "...", "date": "..." } ] }`

POST `/fuel-expenses/fuel`

Request body: `{ "vehicle": "<id>", "trip": "<id optional>", "liters": 40, "costPerLiter": 89.5, "cost": 3580, "station": "HP Petrol Pump" }` Response `201`: `{ "success": true, "data": { ... } }`

GET `/fuel-expenses/expenses?vehicle=<id>&expenseType=toll`

Response `200`: `{ "success": true, "count": 3, "data": [ { "expenseType": "toll", "amount": 450, ... } ] }`

POST `/fuel-expenses/expenses`

Request body: `{ "vehicle": "<id>", "trip": "<id optional>", "expenseType": "toll", "amount": 450, "description": "Mumbai-Pune Expressway" }` Valid `expenseType`: `toll`, `insurance`, `tax`, `fine`, `parking`, `other`. Response `201`: `{ "success": true, "data": { ... } }`

---

7. Maintenance — `/api/v1/maintenance`

Method	Path	Access
GET	`/`, `/:id`	Any authenticated
POST	`/`	`fleet_manager`, `safety_officer`
PUT	`/:id`	`fleet_manager`, `safety_officer`
PUT	`/:id/close`	`fleet_manager`, `safety_officer`

GET `/maintenance?status=open&vehicle=<id>`

Response `200`: `{ "success": true, "count": 2, "data": [{ "vehicle": {"name":"...","registrationNumber":"...","status":"in_shop"}, "maintenanceType": "preventive", "status": "open",

`... } ] }`

POST `/maintenance`

Request body: `{ "vehicle": "<id>", "description": "10,000 km service", "maintenanceType": "preventive", "cost": 4500 }`

Response `201`:

```
{ "success": true, "message": "Maintenance record created. Vehicle status set to in_shop.", "data": { "status "open", ... } }
```

Error `400` (BR-9): `{ "success": false, "message": "BR-9: Vehicle 'Tata 1613 Truck' is currently on a trip. Cannot open maintenance record." }` — same pattern for `retired` vehicles. Error `404`: `{ "success": false, "message": "Vehicle not found" }`

PUT `/maintenance/:id/close`

Request body (optional): `{ "cost": 4800, "notes": "Replaced brake pads" }` Response `200`:

```
{ "success": true, "message": "Maintenance closed. Vehicle restored to available.", "data": { "status": "closed", "dateEnd": "...", ... } }
```

Error `400`: `{ "success": false, "message": "Maintenance record is already closed" }` *Note: vehicle is only restored to `available` if it was not independently marked `retired`.*

PUT `/maintenance/:id`

Request body: any subset of `{ description, maintenanceType, cost, notes }`. Response `200`: `{ "success": true, "data": { ... } }`

---

8. Reports — `/api/v1/reports`

Method	Path	Access	Cache
GET	`/dashboard`	Any authenticated	300s
GET	`/fuel-efficiency`	Any authenticated	300s
GET	`/operational-cost`	Any authenticated	300s
GET	`/driver-performance`	`financial_analyst`, `fleet_manager`	300s
GET	`/export/trips`	`financial_analyst`, `fleet_manager`	—

GET `/reports/dashboard`

Response `200`:


```
{
  "success": true,
  "data": {
    "vehicles": { "total": 24, "available": 18, "onTrip": 4, "inShop": 1, "retired": 1 },
    "drivers": { "total": 30, "available": 22, "suspended": 1 },
    "trips": { "total": 340, "completed": 289, "dispatched": 12, "cancelled": 8 },
    "maintenance": { "open": 1 },
    "financial": {
      "totalRevenue": "1845600.00",
      "totalFuelCost": "312400.00",
      "totalExpenses": "58900.00",
      "totalMaintenanceCost": "94200.00",
      "totalCosts": "465500.00",
      "netProfit": "1380100.00",
      "fuelEfficiencyKmPerL": 6.42,
      "roi": 296.46
```

```
        }
      }
    }
```

GET `/reports/fuel-efficiency`

Response `200` : `{ "success": true, "data": [ { "vehicleName": "...", "regNumber": "...", "totalDistance": 4820, "totalFuel": 610, "tripCount": 34, "kmPerLiter": 7.9 } ] }` (sorted best-to-worst)

GET `/reports/operational-cost`

Response `200` : `{ "success": true, "data": [ { "vehicleId": "...", "name": "...", "registrationNumber": "...", "fuelCost": 45200, "expenseCost": 3100, "maintenanceCost": 8900, "totalCost": 57200, "revenue": 210000, "profit": 152800 } ] }`

GET `/reports/driver-performance`

Response `200` : `{ "success": true, "data": [ { "driverId": "...", "name": "...", "licenseNumber": "...", "totalTrips": 42, "totalDistance": 6200, "totalRevenue": 310000, "totalFuel": 780 } ] }` (sorted by revenue desc)

GET `/reports/export/trips`

Streams a `text/csv` file (`trips_report.csv`) of all completed trips with columns: `tripCode, vehicle, registration, driver, source, destination, cargoWeight, actualDistance, fuelConsumed, revenue, completedAt`. Not JSON — triggers a file download.

---

9. Notifications — `/api/v1/notifications`

Method	Path	Access
GET	`/`	Any authenticated
PUT	`/:id/read`	Any authenticated

GET `/notifications`

Returns the caller's personal notifications **plus** any broadcast notifications (no `userId`), most recent first, capped at 50.
Response `200` : `{ "success": true, "count": 7, "data": [ { "title": "Maintenance Due Soon", "message": "...", "type": "warning", "read": false, "createdAt": "..." } ] }`

PUT `/notifications/:id/read`

Response `200` : `{ "success": true, "data": { "read": true, ... } }` Error `404` : `{ "success": false, "message": "Notification not found" }`

---

10. AI (Stub) — `/api/v1/ai`

⚠ **Not production logic, and not authenticated** — mounted with no `protect` / `authorize` middleware at all, so both endpoints are reachable without a JWT. They return hardcoded/mocked data with an artificial delay.

POST `/ai/optimize-route`

Request body: `{ "source": "Pune", "destination": "Delhi" }` Response `200` (after ~1s): `{ "success": true, "route": [ {"lat":18.52,"lng":73.86,"name":"Pune"}, ... ], "distanceSaved": 34, "etaSaved": "45 mins" }`

POST `/ai/assistant`

Request body: `{ "message": "any text" }` Response `200` (after ~0.8s): `{ "success": true, "reply": "I am the TransitOps AI. I've analyzed the fleet operations, and there are 2 vehicles needing immediate maintenance. Would you like me to schedule them?" }` — the reply is a fixed string regardless of input.

---

11. System / Meta Endpoints

Method	Path	Purpose
GET	`/api/v1/health`	Liveness check — returns `{ status, timestamp }`
GET	`/api-docs`	Swagger UI (auto-generated from `swagger-jsdoc` annotations)
GET	`/status`	`express-status-monitor` live dashboard (CPU, memory, response times)

12. Known API-Level Discrepancies

#	Issue
1	Two separate `authorize()` implementations exist (`middleware/authMiddleware.js` and `middleware/roleMiddleware.js`) with equivalent 403-on-role-mismatch behavior — Vehicle/Driver routes import from `authMiddleware`, while Trip/Maintenance/Fuel-Expense/Report routes import from `roleMiddleware`. Functionally identical today, but a maintenance risk if they drift.
2	Driver `status` enum differs between the request-layer Zod validator (`on_leave`) and the persisted Mongoose schema (`off_duty`) — see Backend Schema doc §7, item 1.
3	`/ai/*` endpoints have **no `protect` or `authorize()` middleware at all** — they are fully unauthenticated and callable by anyone with network access to the API, and return mocked, not real, data. Must not be relied on or exposed publicly until secured.

Companion documents: PRD, TRD, Backend Schema, SRS, ER Diagram, UML Class Diagram, Trip Lifecycle Workflow, UI/UX Theme Guide.